

ASSIGNMENT 17.3

Name: K.Sanjana

Roll.no:2303a52139

Batchno:41

Task1: Employee Dataset Cleaning

Prompt: # Create an employee information dataset and handle missing values in salary, designation, and experience. Remove duplicate employee IDs and standardised text values(job values in lowercase), encode categorical features(designation, employment_type).give clean dataset for hr analytics and prediction tasks in the output.

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder

# 1. Create a sample employee information dataset
data = {
    'employee_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 101, 111, 112],
    'salary': [50000, 60000, np.nan, 75000, 55000, 50000, 62000, np.nan, 70000, 80000, 50000, 65000, 72000],
    'designation': ['Software Engineer', 'Data Scientist', 'Project Manager', 'Software Engineer', 'HR Specialist', 'Lead Engineer', 'Data Analyst', np.nan, 'HR Specialist', 'Software Engineer', 'Software Engineer', 'Product Manager'],
    'experience': [3, 5, 8, 4, 2, 10, 3, np.nan, 6, 5, 3, 7, 4],
    'employment_type': ['Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-time', 'Full-time', 'Full-time', 'Contract', 'Full-time', 'Full-time', 'Full-time', 'Full-time'],
    'gender': ['Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Male']
}

df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Make a copy to work on
clean_df = df.copy()

# 2. Handle missing values
# Impute 'salary' with median
median_salary = clean_df['salary'].median()
clean_df['salary'] = clean_df['salary'].fillna(median_salary)

# Impute 'designation' with mode
mode_designation = clean_df['designation'].mode()[0]
clean_df['designation'] = clean_df['designation'].fillna(mode_designation)

# Impute 'experience' with median
median_experience = clean_df['experience'].median()
clean_df['experience'] = clean_df['experience'].fillna(median_experience)

# 3. Remove duplicate employee IDs
# Keep the first occurrence
clean_df.drop_duplicates(subset=['employee_id'], keep='first', inplace=True)

# 4. Standardize text values (e.g., 'designation' to lowercase)
clean_df['designation'] = clean_df['designation'].str.lower()
```

```

# 5. Encode categorical features (designation, employment_type)
# Initialize OneHotEncoder
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)

# Fit and transform the categorical features
categorical_features = ['designation', 'employment_type']
encoded_features = encoder.fit_transform(clean_df[categorical_features])

# Create a DataFrame with the encoded features
encoded_df = pd.DataFrame(encoded_features, columns=encoder.get_feature_names_out(categorical_features))

# Reset index of clean_df before concatenating to avoid alignment issues
clean_df = clean_df.reset_index(drop=True)

# Concatenate the encoded features with the original DataFrame (dropping original categorical columns)
clean_df = pd.concat([clean_df.drop(columns=categorical_features), encoded_df], axis=1)

print("\nCleaned DataFrame for HR analytics and prediction tasks:")
print(clean_df)

```

Output:

```

Original DataFrame:
   employee_id  salary  designation  experience  employment_type  gender
0          101  50000.0  Software Engineer         3.0      Full-time   Male
1          102  60000.0   Data Scientist         5.0      Full-time  Female
2          103    NaN    Project Manager         8.0      Contract   Male
3          104  75000.0  Software Engineer         4.0      Full-time  Female
4          105  55000.0   HR Specialist         2.0      Part-time  Female
5          106  90000.0   Lead Engineer        10.0      Full-time   Male
6          107  62000.0   Data Analyst         3.0      Full-time   Male
7          108    NaN      NaN      NaN      Contract  Female
8          109  70000.0   HR Specialist         6.0      Full-time  Female
9          110  80000.0  Software Engineer         5.0      Full-time   Male
10         101  50000.0  Software Engineer         3.0      Full-time   Male
11         111  65000.0   Product Manager         7.0      Full-time  Female
12         112  72000.0   Data Scientist         4.0      Full-time   Male

Cleaned DataFrame for HR analytics and prediction tasks:
   employee_id  salary  experience  gender  designation_data analyst  designation_data scientist  designation_hr specialist
0          101  50000.0         3.0   Male              0.0              0.0              0.0
1          102  60000.0         5.0  Female              0.0              1.0              0.0
2          103  65000.0         8.0   Male              0.0              0.0              0.0
3          104  75000.0         4.0  Female              0.0              0.0              0.0
4          105  55000.0         2.0  Female              0.0              0.0              1.0
5          106  90000.0        10.0   Male              0.0              0.0              0.0
6          107  62000.0         3.0   Male              1.0              0.0              0.0
7          108  65000.0         4.5  Female              0.0              0.0              0.0
8          109  70000.0         6.0  Female              0.0              0.0              1.0
9          110  80000.0         5.0   Male              0.0              0.0              0.0
10         101  50000.0         7.0  Female              0.0              0.0              0.0
11         112  72000.0         4.0   Male              0.0              1.0              0.0

```

Justification

This code generates and cleans a sample employee dataset by handling missing values, removing duplicates, standardizing text, and encoding categorical features, making it ready for HR analytics.

TASK2: Student Performance Data Preprocessing

Prompt: Create a student dataset and Preprocess student performance data using AI assistance. Convert the exam_date column into a datetime format. Extract new features (month, semester, day), normalise total_marks, detect and handle outliers in marks using IQR or Z-score. Give a transformed dataset with time-based academic features and normalised marks as outputs.

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder

# Set pandas display options for neat output
pd.set_option('display.max_columns', None)
pd.set_option('display.width', 1000) # Adjust based on screen width

# 1. Create a sample employee information dataset
data = {
    'employee_id': [101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 101, 111, 112],
    'salary': [50000, 60000, np.nan, 75000, 55000, 90000, 62000, np.nan, 70000, 80000, 50000, 65000, 72000],
    'designation': ['Software Engineer', 'Data Scientist', 'Project Manager', 'Software Engineer', 'HR Specialist', 'Lead Engineer', 'Data Analyst', 'Software Engineer', 'Data Scientist', 'Project Manager', 'Software Engineer', 'HR Specialist', 'Lead Engineer'],
    'experience': [3, 5, 8, 4, 2, 10, 3, np.nan, 6, 5, 3, 7, 4],
    'employment_type': ['Full-time', 'Full-time', 'Contract', 'Full-time', 'Part-time', 'Full-time', 'Full-time', 'Contract', 'Full-time', 'Full-time', 'Full-time', 'Contract', 'Full-time'],
    'gender': ['Male', 'Female', 'Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Female', 'Male', 'Male', 'Female', 'Male']
}
df = pd.DataFrame(data)

print("Original DataFrame:")
print(df)

# Make a copy to work on
clean_df = df.copy()

# 2. Handle missing values
# Impute 'salary' with median
median_salary = clean_df['salary'].median()
clean_df['salary'] = clean_df['salary'].fillna(median_salary)

# Impute 'designation' with mode
mode_designation = clean_df['designation'].mode()[0]
clean_df['designation'] = clean_df['designation'].fillna(mode_designation)

# Impute 'experience' with median
median_experience = clean_df['experience'].median()
clean_df['experience'] = clean_df['experience'].fillna(median_experience)
```

```

import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# Create a sample student performance dataset
data = {
    'student_id': [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15],
    'exam_date': [
        '2023-01-15', '2023-03-20', '2023-05-10', '2023-02-28', '2023-04-05',
        '2023-09-12', '2023-11-01', '2023-10-25', '2023-07-07', '2023-08-18',
        '2023-01-22', '2023-03-01', '2023-05-20', '2023-02-10', '2023-04-12'
    ],
    'course': [
        'Math', 'Science', 'History', 'Math', 'Science',
        'History', 'Math', 'Science', 'History', 'Math',
        'Science', 'History', 'Math', 'Science', 'History'
    ],
    'total_marks': [
        85, 92, 78, 95, 60, # Example of an outlier (low)
        88, 70, 91, 80, 93,
        105, # Example of an outlier (high)
        82, 75, 89, 65
    ],
    'attendance_percentage': [
        90, 95, 80, 98, 75,
        92, 88, 96, 85, 99,
        70, 91, 83, 94, 78
    ],
    'gender': [
        'Male', 'Female', 'Male', 'Female', 'Male',
        'Female', 'Male', 'Female', 'Male', 'Female',
        'Male', 'Female', 'Male', 'Female', 'Male'
    ]
}
df_students = pd.DataFrame(data)

print("Original Student DataFrame:")
print(df_students)

```

```
# Convert 'exam_date' to datetime format
df_students['exam_date'] = pd.to_datetime(df_students['exam_date'])

# Extract new time-based features
df_students['exam_month'] = df_students['exam_date'].dt.month
df_students['exam_day'] = df_students['exam_date'].dt.day
df_students['exam_day_of_week'] = df_students['exam_date'].dt.dayofweek # Monday=0, Sunday=6

# Define semesters (example: Semester 1: Jan-May, Semester 2: Aug-Dec)
def get_semester(month):
    if 1 <= month <= 5:
        return 1
    elif 8 <= month <= 12:
        return 2
    else:
        return np.nan # Or define as another 'break' semester

df_students['semester'] = df_students['exam_month'].apply(get_semester)

print("DataFrame with time-based features:")
display(df_students.head())
```

```
# Normalize 'total_marks' using Min-Max Scaling
scaler = MinMaxScaler()
df_students['normalized_marks'] = scaler.fit_transform(df_students[['total_marks']])

print("DataFrame with normalized marks:")
display(df_students.head())
```

Output:

*** Original Student DataFrame:

	student_id	exam_date	course	total_marks	attendance_percentage	gender
0	1	2023-01-15	Math	85	90	Male
1	2	2023-03-20	Science	92	95	Female
2	3	2023-05-10	History	78	80	Male
3	4	2023-02-28	Math	95	98	Female
4	5	2023-04-05	Science	60	75	Male

DataFrame with time-based features:

	student_id	exam_date	course	total_marks	attendance_percentage	gender	exam_month	exam_day	exam_day_of_week	semester
0	1	2023-01-15	Math	85	90	Male	1	15	6	1.0
1	2	2023-03-20	Science	92	95	Female	3	20	0	1.0
2	3	2023-05-10	History	78	80	Male	5	10	2	1.0
3	4	2023-02-28	Math	95	98	Female	2	28	1	1.0
4	5	2023-04-05	Science	60	75	Male	4	5	2	1.0

iFrame with normalized marks:											
student_id	exam_date	course	total_marks	attendance_percentage	gender	exam_month	exam_day	exam_day_of_week	semester	normalized_marks	
1	2023-01-15	Math	85	90	Male	1	15	6	1.0	0.555556	
2	2023-03-20	Science	92	95	Female	3	20	0	1.0	0.711111	
3	2023-05-10	History	78	80	Male	5	10	2	1.0	0.400000	
4	2023-02-28	Math	95	98	Female	2	28	1	1.0	0.777778	
5	2023-04-05	Science	60	75	Male	4	5	2	1.0	0.000000	

Justification:

This code preprocesses student performance data by creating a sample dataset, extracting time-based features from exam dates, normalizing total marks, and detecting/handling outliers using the IQR method, preparing the data for analysis.

Task 3: Student Health Data Preparation

Prompt: Clean and preprocess student health records using AI scripts. Handle missing values in height, weight, BMI, Encode categorical features (medical_condition, fitness_status), Scale numerical features using standard scaling, Split dataset into training and testing sets Give a preprocessed dataset ready for predictive health analysis as output.

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.model_selection import train_test_split

# 1. Create a sample student health records dataset
data_health = {
    'student_id': [201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215],
    'age': [16, 17, 15, 16, 18, 17, 16, 15, 17, 18, 16, 17, 15, 16, 18],
    'height_cm': [165, 172, 160, 168, 175, 170, 163, np.nan, 171, 178, 166, 173, 161, 169, 176],
    'weight_kg': [55, 62, 50, 58, 65, np.nan, 53, 48, 60, 68, 56, 63, 51, 59, np.nan],
    'bmi': [20.2, 20.9, 19.5, 20.5, 21.2, 21.5, np.nan, 19.0, 20.6, 21.3, 20.3, 21.0, 19.7, 20.4, 21.1],
    'blood_pressure_systolic': [110, 115, 108, 112, 120, 118, 110, 105, 114, 122, 111, 116, 109, 113, 121],
    'blood_pressure_diastolic': [70, 75, 68, 72, 80, 78, 70, 65, 74, 82, 71, 76, 69, 73, 81],
    'medical_condition': ['None', 'Asthma', 'None', 'Allergies', 'None', 'Asthma', 'None', 'Allergies', 'None', 'Diabetes', 'None', 'Asthma', 'None', 'Allergies', 'None'],
    'fitness_status': ['Good', 'Excellent', 'Average', 'Good', 'Excellent', 'Good', 'Average', 'Good', 'Excellent', 'Good', 'Average', 'Good', 'Excellent', 'Good', 'Average'],
    'sleep_hours': [7, 8, 7, 7, 8, 6, 7, 8, 7, 8, 7, 6, 8, 7, 7, 8],
    'activity_level': [3, 4, 2, 3, 4, 3, 2, 3, 4, 3, 2, 4, 3, 3, 4]
}
df_health = pd.DataFrame(data_health)

print("Original Student Health Records DataFrame:")
display(df_health.head())

# Make a copy to work on
clean_df_health = df_health.copy()
```

```
# 2. Handle missing values in height, weight, BMI
# Impute with median for numerical columns
median_height = clean_df_health['height_cm'].median()
clean_df_health['height_cm'] = clean_df_health['height_cm'].fillna(median_height)

median_weight = clean_df_health['weight_kg'].median()
clean_df_health['weight_kg'] = clean_df_health['weight_kg'].fillna(median_weight)

median_bmi = clean_df_health['bmi'].median()
clean_df_health['bmi'] = clean_df_health['bmi'].fillna(median_bmi)

print("DataFrame after handling missing values:")
display(clean_df_health.head())
```

```
# 3. Encode categorical features (medical_condition, fitness_status)
encoder = OneHotEncoder(handle_unknown='ignore', sparse_output=False)
categorical_features_health = ['medical_condition', 'fitness_status']
encoded_features_health = encoder.fit_transform(clean_df_health[categorical_features_health])

encoded_df_health = pd.DataFrame(encoded_features_health, columns=encoder.get_feature_names_out(categorical_features_health))

# Reset index before concatenating to avoid alignment issues
clean_df_health = clean_df_health.reset_index(drop=True)

# Concatenate the encoded features with the original DataFrame (dropping original categorical columns)
clean_df_health = pd.concat([clean_df_health.drop(columns=categorical_features_health), encoded_df_health], axis=1)

print("DataFrame after encoding categorical features:")
display(clean_df_health.head())
```

```
# 4. Scale numerical features using standard scaling
# Identify numerical features to scale (excluding 'student_id' and encoded features)
numerical_features_health = [
    'age', 'height_cm', 'weight_kg', 'bmi', 'blood_pressure_systolic',
    'blood_pressure_diastolic', 'sleep_hours', 'activity_level'
]

scaler_health = StandardScaler()
clean_df_health[numerical_features_health] = scaler_health.fit_transform(clean_df_health[numerical_features_health])

print("DataFrame after scaling numerical features:")
display(clean_df_health.head())
```

```
# 5. Split dataset into training and testing sets
# For demonstration, let's assume 'blood_pressure_systolic' is a target variable for a predictive task
# In a real scenario, you would define your target based on the problem statement.

X = clean_df_health.drop(columns=['student_id', 'blood_pressure_systolic', 'blood_pressure_diastolic']) # Drop student_id and target related columns
y_systolic = clean_df_health['blood_pressure_systolic']
y_diastolic = clean_df_health['blood_pressure_diastolic']

# Split for systolic pressure prediction
X_train_systolic, X_test_systolic, y_train_systolic, y_test_systolic = train_test_split(
    X, y_systolic, test_size=0.2, random_state=42
)

# Split for diastolic pressure prediction
X_train_diastolic, X_test_diastolic, y_train_diastolic, y_test_diastolic = train_test_split(
    X, y_diastolic, test_size=0.2, random_state=42
)

print("\nDataset split into training and testing sets.")
print(f"X_train_systolic shape: {X_train_systolic.shape}")
print(f"X_test_systolic shape: {X_test_systolic.shape}")
print(f"y_train_systolic shape: {y_train_systolic.shape}")
print(f"y_test_systolic shape: {y_test_systolic.shape}")

print("\nPreprocessed DataFrame ready for predictive health analysis (first 5 rows):")
display(clean_df_health.head())
```

Output:

DataFrame after handling missing values:

	student_id	age	height_cm	weight_kg	bmi	blood_pressure_systolic	blood_pressure_diastolic	medical_condition	fitness_status	sleep_hours
0	201	16	165.0	55.0	20.2	110	70	None	Good	7
1	202	17	172.0	62.0	20.9	115	75	Asthma	Excellent	8
2	203	15	160.0	50.0	19.5	108	68	None	Average	7
3	204	16	168.0	58.0	20.5	112	72	Allergies	Good	7
4	205	18	175.0	65.0	21.2	120	80	None	Excellent	8

Original Student Health Records DataFrame:

	student_id	age	height_cm	weight_kg	bmi	blood_pressure_systolic	blood_pressure_diastolic	medical_condition	fitness_status	sleep_hours
0	201	16	165.0	55.0	20.2	110	70	None	Good	7
1	202	17	172.0	62.0	20.9	115	75	Asthma	Excellent	8
2	203	15	160.0	50.0	19.5	108	68	None	Average	7
3	204	16	168.0	58.0	20.5	112	72	Allergies	Good	7
4	205	18	175.0	65.0	21.2	120	80	None	Excellent	8

DataFrame after encoding categorical features:

	student_id	age	height_cm	weight_kg	bmi	blood_pressure_systolic	blood_pressure_diastolic	sleep_hours	activity_level	medical_condition_A
0	201	16	165.0	55.0	20.2	110	70	7	3	
1	202	17	172.0	62.0	20.9	115	75	8	4	
2	203	15	160.0	50.0	19.5	108	68	7	2	
3	204	16	168.0	58.0	20.5	112	72	7	3	
4	205	18	175.0	65.0	21.2	120	80	8	4	

DataFrame after scaling numerical features:

	student_id	age	height_cm	weight_kg	bmi	blood_pressure_systolic	blood_pressure_diastolic	sleep_hours	activity_level	medical_c
0	201	-0.455661	-0.790903	-0.478105	-0.465608	-0.739379	-0.739379	-0.306186	-0.185695	
1	202	0.520756	0.559419	0.809101	0.563631	0.287536	0.287536	1.224745	1.207020	
2	203	-1.432078	-1.755418	-1.397537	-1.494847	-1.150145	-1.150145	-0.306186	-1.578410	
3	204	-0.455661	-0.212193	0.073555	-0.024506	-0.328613	-0.328613	-0.306186	-0.185695	
4	205	1.497172	1.138128	1.360760	1.004733	1.314452	1.314452	1.224745	1.207020	

Dataset split into training and testing sets.
X_train_systolic shape: (12, 13)
X_test_systolic shape: (3, 13)
y_train_systolic shape: (12,)
y_test_systolic shape: (3,)

Preprocessed DataFrame ready for predictive health analysis (first 5 rows):

	student_id	age	height_cm	weight_kg	bmi	blood_pressure_systolic	blood_pressure_diastolic	sleep_hours	activity_level	medical_c
0	201	-0.455661	-0.790903	-0.478105	-0.465608	-0.739379	-0.739379	-0.306186	-0.185695	
1	202	0.520756	0.559419	0.809101	0.563631	0.287536	0.287536	1.224745	1.207020	
2	203	-1.432078	-1.755418	-1.397537	-1.494847	-1.150145	-1.150145	-0.306186	-1.578410	
3	204	-0.455661	-0.212193	0.073555	-0.024506	-0.328613	-0.328613	-0.306186	-0.185695	
4	205	1.497172	1.138128	1.360760	1.004733	1.314452	1.314452	1.224745	1.207020	

Justification:

This code preprocesses student health records by handling missing values, encoding categorical features, scaling numerical features, and splitting the dataset into training and testing sets, making it ready for predictive health analysis.

Task4: Real-Time Application: Ride-Sharing Data Cleaning

Prompt: Create a ride-sharing company has trip data with errors..Standardize pickup and drop locations, Fill missing fare amounts with median values, Remove duplicate ride entries, Normalize driver ratings, Generate a summary of data improvements Give a clean dataset ready for operational analysis as output.

Code:

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

# 1. Create a sample ride-sharing trip dataset
data_rideshare = {
    'trip_id': [1001, 1002, 1003, 1004, 1005, 1006, 1007, 1008, 1009, 1010, 1001, 1011, 1012],
    'driver_id': [1, 2, 3, 1, 4, 5, 2, 6, 3, 7, 1, 8, 9],
    'pickup_location': [
        'Downtown ',
        'Airport',
        'suburb A',
        'downtown',
        'University ',
        'Airport',
        'Suburb A',
        'Financial District',
        'suburb B',
        'downtown',
        'Downtown',
        'Airport',
        'University'
    ],
    'dropoff_location': [
        'Financial District',
        'suburb C',
        'Downtown',
        'Airport',
        'Downtown',
        'Suburb A',
        'Financial District',
        'Airport',
        'Downtown',
        'Suburb C',
        'Financial District',
        'Suburb B',
        'Downtown'
    ],
    'fare_amount': [15.50, 25.00, np.nan, 20.00, 12.75, 30.00, 18.20, np.nan, 16.00, 22.50, 15.50, 28.00, 14.00],
    'trip_duration_minutes': [20, 35, 25, 30, 15, 40, 22, 38, 18, 32, 20, 30, 20],
    'distance_miles': [5.2, 12.1, 7.5, 9.8, 3.1, 15.0, 6.5, 10.5, 4.8, 11.2, 5.2, 8.5, 4.0],
}

# 2. Standardize pickup and drop locations
# Convert to lowercase and strip whitespace
clean_df_rideshare['pickup_location'] = clean_df_rideshare['pickup_location'].str.lower().str.strip()
clean_df_rideshare['dropoff_location'] = clean_df_rideshare['dropoff_location'].str.lower().str.strip()

print("DataFrame after standardizing locations:")
display(clean_df_rideshare.head())
```

```

# 3. Fill missing fare amounts with median values
median_fare = clean_df_rideshare['fare_amount'].median()
clean_df_rideshare['fare_amount'] = clean_df_rideshare['fare_amount'].fillna(median_fare)

print("DataFrame after filling missing fare amounts:")
display(clean_df_rideshare.head())

# 4. Remove duplicate ride entries
# Assuming (trip_id, driver_id, pickup_location, dropoff_location) uniquely identify a ride before considering duplicates
initial_rows = clean_df_rideshare.shape[0]
clean_df_rideshare.drop_duplicates(subset=['trip_id', 'driver_id', 'pickup_location', 'dropoff_location'], keep='first', inplace=True)
duplicates_removed = initial_rows - clean_df_rideshare.shape[0]

print(f"Removed {duplicates_removed} duplicate ride entries.")
print("DataFrame after removing duplicate ride entries:")
display(clean_df_rideshare.head())

# 5. Normalize driver ratings
scaler_rating = MinMaxScaler()
clean_df_rideshare['normalized_driver_rating'] = scaler_rating.fit_transform(clean_df_rideshare[['driver_rating']])

print("DataFrame after normalizing driver ratings:")
display(clean_df_rideshare.head())

# 6. Generate a summary of data improvements
print("\n--- Data Improvement Summary ---")
print(f"- Standardized 'pickup_location' and 'dropoff_location' to lowercase and stripped whitespace.")
print(f"- Missing 'fare_amount' values imputed with median: {median_fare:.2f}.")
print(f"- Removed {duplicates_removed} duplicate ride entries based on (trip_id, driver_id, pickup_location, dropoff_location).")
print(f"- Normalized 'driver_rating' to a 0-1 scale, available in 'normalized_driver_rating'.")
print("\nCleaned DataFrame ready for operational analysis (first 5 rows):")
display(clean_df_rideshare.head())

```

Output:

Original Ride-Sharing DataFrame:								
	trip_id	driver_id	pickup_location	dropoff_location	fare_amount	trip_duration_minutes	distance_miles	driver_rating
0	1001	1	Downtown	Financial District	15.50	20	5.2	4.8
1	1002	2	Airport	suburb C	25.00	35	12.1	4.5
2	1003	3	suburb A	Downtown	NaN	25	7.5	3.9
3	1004	1	downtown	Airport	20.00	30	9.8	4.9
4	1005	4	University	Downtown	12.75	15	3.1	4.2
5	1006	5	Airport	Suburb A	30.00	40	15.0	5.0
6	1007	2	Suburb A	Financial District	18.20	22	6.5	4.6
7	1008	6	Financial District	Airport	NaN	38	10.5	4.0
8	1009	3	suburb B	Downtown	16.00	18	4.8	4.7
9	1010	7	downtown	Suburb C	22.50	32	11.2	4.3
10	1001	1	Downtown	Financial District	15.50	20	5.2	4.8
11	1011	8	Airport	Suburb B	28.00	30	8.5	4.5
12	1012	9	University	Downtown	14.00	20	4.0	4.1

DataFrame after standardizing locations:

trip_id	driver_id	pickup_location	dropoff_location	fare_amount	trip_duration_minutes	distance_miles	driver_rating	
0	1001	1	downtown	financial district	15.50	20	5.2	4.8
1	1002	2	airport	suburb c	25.00	35	12.1	4.5
2	1003	3	suburb a	downtown	NaN	25	7.5	3.9
3	1004	1	downtown	airport	20.00	30	9.8	4.9
4	1005	4	university	downtown	12.75	15	3.1	4.2

DataFrame after filling missing fare amounts:

trip_id	driver_id	pickup_location	dropoff_location	fare_amount	trip_duration_minutes	distance_miles	driver_rating	
0	1001	1	downtown	financial district	15.50	20	5.2	4.8
1	1002	2	airport	suburb c	25.00	35	12.1	4.5
2	1003	3	suburb a	downtown	18.20	25	7.5	3.9
3	1004	1	downtown	airport	20.00	30	9.8	4.9
4	1005	4	university	downtown	12.75	15	3.1	4.2

Removed 1 duplicate ride entries.

DataFrame after removing duplicate ride entries:

trip_id	driver_id	pickup_location	dropoff_location	fare_amount	trip_duration_minutes	distance_miles	driver_rating	
0	1001	1	downtown	financial district	15.50	20	5.2	4.8
1	1002	2	airport	suburb c	25.00	35	12.1	4.5
2	1003	3	suburb a	downtown	18.20	25	7.5	3.9
3	1004	1	downtown	airport	20.00	30	9.8	4.9
4	1005	4	university	downtown	12.75	15	3.1	4.2

--- Data Improvement Summary ---

- Standardized 'pickup_location' and 'dropoff_location' to lowercase and stripped whitespace.

- Missing 'fare_amount' values imputed with median: 18.20.

- Removed 1 duplicate ride entries based on (trip_id, driver_id, pickup_location, dropoff_location).

- Normalized 'driver_rating' to a 0-1 scale, available in 'normalized_driver_rating'.

Cleaned DataFrame ready for operational analysis (first 5 rows):

trip_id	driver_id	pickup_location	dropoff_location	fare_amount	trip_duration_minutes	distance_miles	driver_rating	normalized_driver_rating	
0	1001	1	downtown	financial district	15.50	20	5.2	4.8	0.8182
1	1002	2	airport	suburb c	25.00	35	12.1	4.5	0.5455
2	1003	3	suburb a	downtown	18.20	25	7.5	3.9	0.0000
3	1004	1	downtown	airport	20.00	30	9.8	4.9	0.9091
4	1005	4	university	downtown	12.75	15	3.1	4.2	0.2727

DataFrame after normalizing driver ratings:

trip_id	driver_id	pickup_location	dropoff_location	fare_amount	trip_duration_minutes	distance_miles	driver_rating	normalized_driver_rating	
0	1001	1	downtown	financial district	15.50	20	5.2	4.8	0.8182
1	1002	2	airport	suburb c	25.00	35	12.1	4.5	0.5455
2	1003	3	suburb a	downtown	18.20	25	7.5	3.9	0.0000
3	1004	1	downtown	airport	20.00	30	9.8	4.9	0.9091
4	1005	4	university	downtown	12.75	15	3.1	4.2	0.2727

Justification:

1. Standardizing Locations: Ensures consistency and accurate grouping by treating semantically identical locations as truly identical.

2. Handling Missing Fare Amounts (Median Imputation): Fills gaps to maintain data completeness and avoids skewing distributions, especially with potential outliers, by using the robust median.
3. Removing Duplicate Entries: Maintains data integrity and prevents overcounting in analyses by ensuring each unique ride record is represented only once.
4. Normalizing Driver Ratings: Scales ratings to a common range (0-1), preventing features with larger numerical values from disproportionately influencing machine learning models.